Ch-04

- A thread is the smallest unit of execution inside a Process.

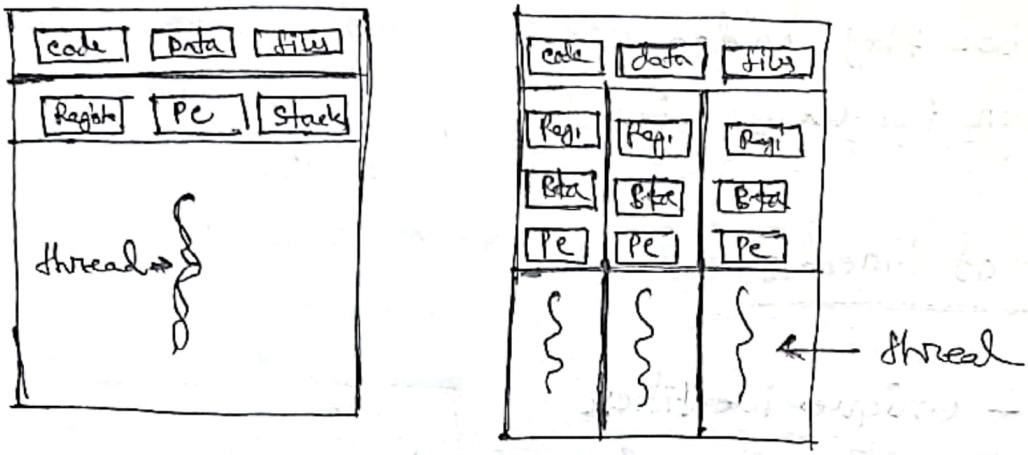
chrome: one thread load the Page
Another Play video
Another handles user input

Basic component of thread:

1. Thread ID — unique identifier
2. Program counter: next instruction counter
3. Registers: Store intermediate values
4. Stack: function calls, local variables

Process	thread
separate memory	Shared memory
communication slow	Fast
creation slow	Fast
~~crash impact~~ one Process crash ≠ others	one thread crash may affect whole Process
High Resource usage	low Resource usage

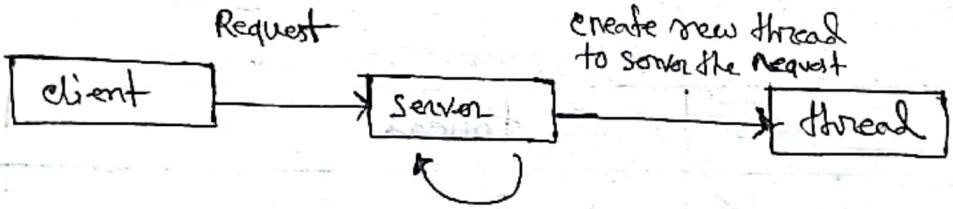
#



Single thread

Multithread Process

Multithread Server Architecture:



benifit:

- Responsiveness: Application remains responsive even if a thread is busy
- Resource sharing: Thread share the same memory and Resources
- Economy (low cost): creating threads is cheaper than Processes.
- Scalability: can run multiple thread on multiple

- Improve performance: Tasks can run concurrently.

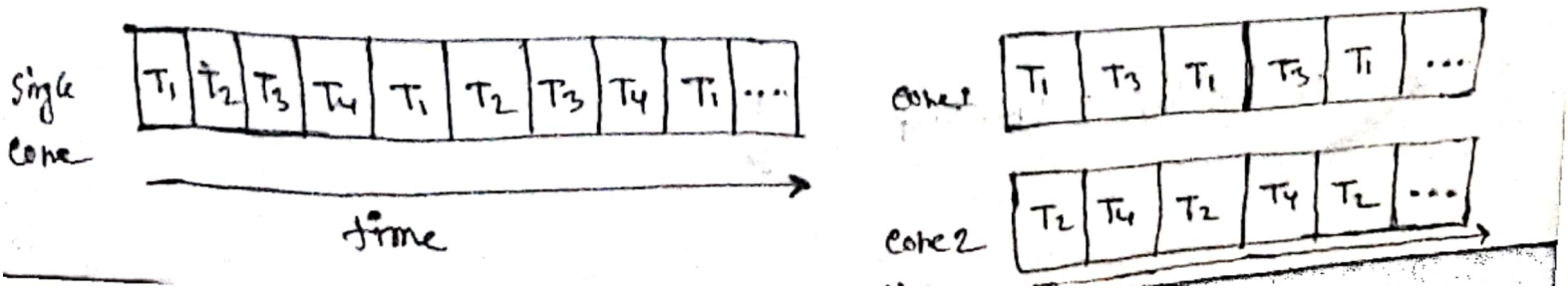
challenges:

- synchronization Problem: multiple thread accessing shared data can cause error.
- Race condition: output depend on thread execution order
- Deadlock: Thread wait forever for each other.
- Starvation: some threads never get cpu time.
- complex debugging: Hard to find bugs.
- context switching overhead: switching between threads cost cpu time.

- multicore / multiprocessor system challenges:

Dividing activities	Parallelism: implies a system can perfo
Balance	more than one task simultaneously.
Data splitting	concurrency support more than one t
Data dependency	making progress, single core
Testing and debugging	

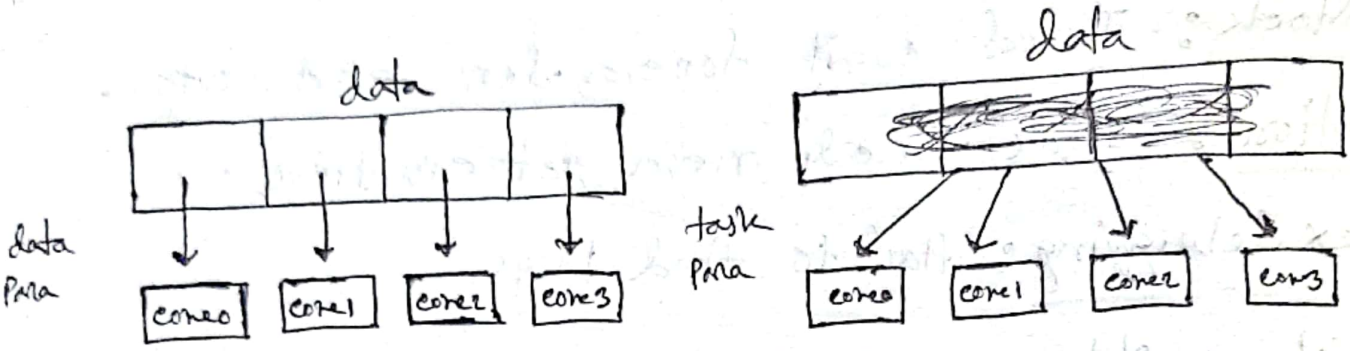
- concurrent execution on single core
- Parallelism on a multicore system



~~multicore~~ Parallelism type:

Data Parallelism: distributes subsets of the same data across m
cores, same operation on each

Task Parallelism: distributing threads across cores, each thr
perform unique operations.



Amdahl's law: is used to measure the maximum improvement
in performance when part of a system is made faster.

$$\text{Speedup} = \frac{1}{(1-P) + \frac{P}{N}}$$

P = Portion of the Program that
be Parallelized

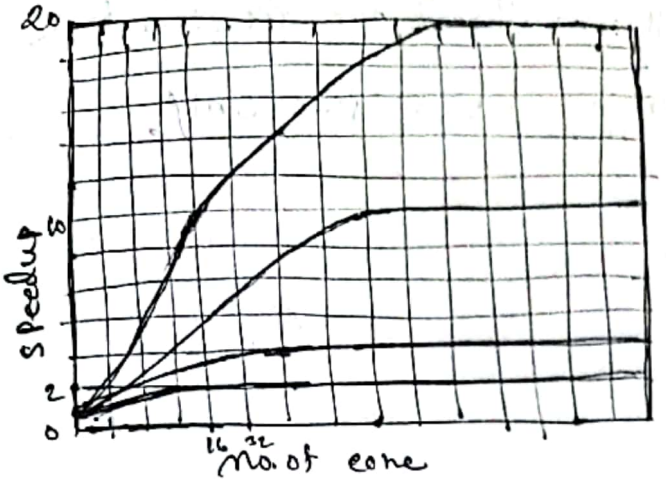
(1-P) = Portion that must rema
sequential

N = no. of thread, core

75%. Parallel, 20% serial, core 2

Speed up 1.6 time

When N infinity: speedup $\frac{1}{P}$

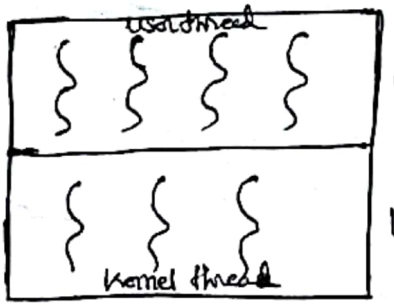


- User thread are manged by a user level thread library. not by the OS kernel.

  Three type: POSIX thread (Pthread)
  windows thread
  Java thread

- Kernel thread are manged by the OS kernel.

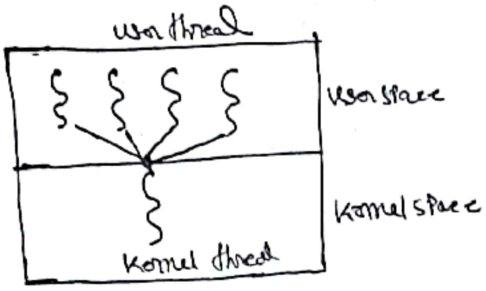
  Linux, Windows, mac OS X, iOS, Android.



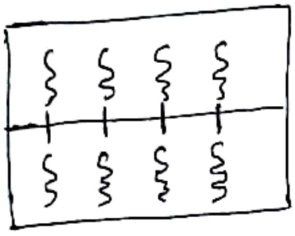
multi threading model: define how user thread are mapped to kernel thread.

three type:

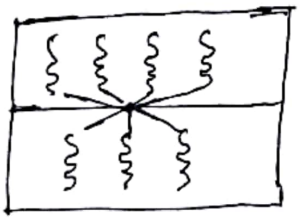
many to one model: All user thread are mapped to a single kernel thread. manage in user space only.



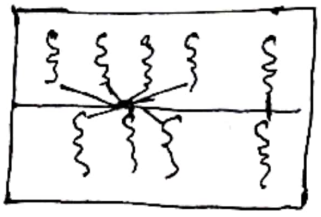
one to one model: Each user thread has its own kernel os schedule each thread independently.



many to many model: multiple user threads are mapp to multiple kernel thread os can schedule them indi



two level model: many to many except that it allows a user thread to be bound to kernel thread



Thread libary Provide Programmer with API for creating and managing thread.

Two Primary Way:
- Libary entirely in user space
- Kernel level libary supported by the OS

Pthread:

Java thread: are managed by the JVM.

Implicit threading: Five method:
- Thread Pools
- Fork Join
- OPENMP
- Grand central Dispatch
- Intel Threading Building Blocks.

implicit threading means the Programmer does not manually create/ manage threads instead the system (compiler, OS) handles it automatical

A thread Pools is a collection of Pre-created threads that wait for task.

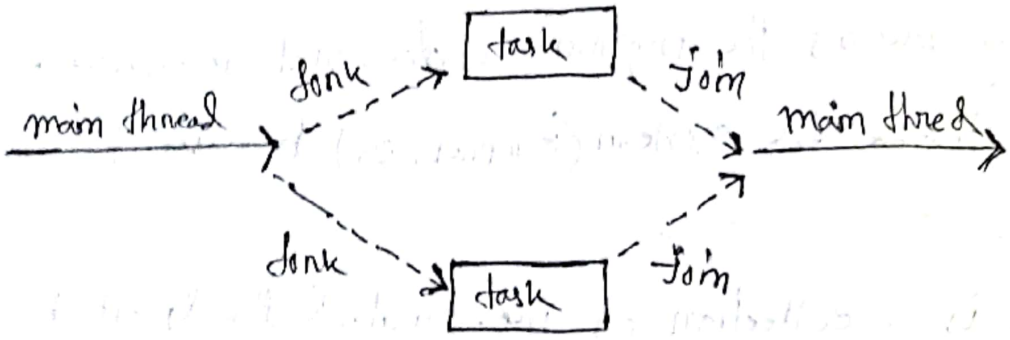
How work:
- Thread are created once and store in Pools
- Task are submitted to a queue.
- Available thread. Picks a task and execute it.

Adv
- Reduce overhead of creating/destroying threads
- Better performance
- Limited no. of concurrent threads

dis
- Fixed pool size may cause waiting

Fork-join – multiple thread (tasks) are forked and then join

How work:
- main task divided into subtask
- subtask run in parallel
- Result are combined.



Adv
- efficient for divide and conquer Pblm
- Automatic Parallelism

Grand signal dispatch: A high level API that manages thread execution automatically using Dispatch queue.

Working:

Developer submit task to queue

System decides:

when to run

which thread to use.

Adv : very simple to use

Optimize by OS

No manual thread handling.

dis less control over threads.

OpenMP

OpenMP is an API for shared memory Parallel Programming using compiler directives. Used in c, c++

How work: add special Program and the compiler automatically create thread.

- Intel threading building block is a c++ template library task based parallelism.

  Work: define task
  TBB runtime schedules them across available

Signal handling is a mechanism used by the OS to notify process that event has occurred.

signal handler is used to process signal

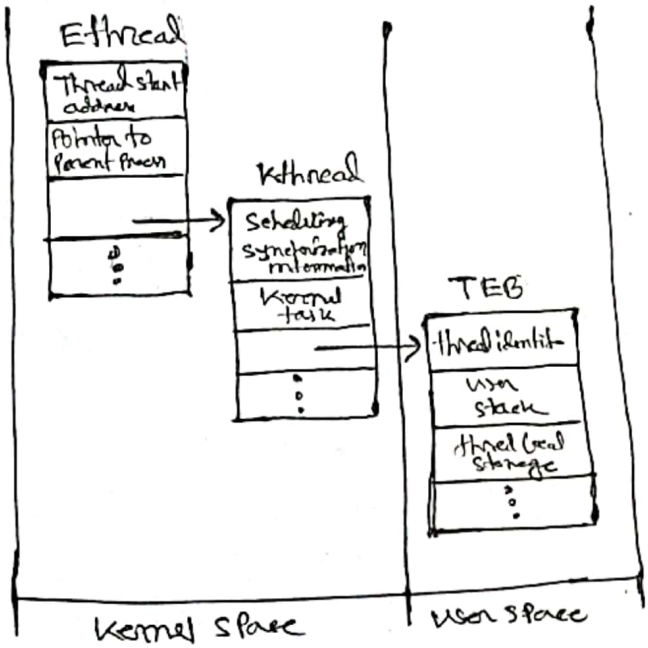
1. Signal is generated by particular event
2. Signal is delivered to a process
3. signal is handled by one two signal handler.
   default, user defined.

Windows thread Representation:

In microsoft windows, thread are treated as seperate kernel object.

Each thread has:

- Thread ID
- Register set
- Stack
- Thread environment



- Thread Shared Process Resource but execute independently.
- Uses one to one threading model.

Linux thread: thread are implemented as Lightweight Processes.

Represent using task struct

create using clone() system call

Thread share resource: memory

flag	meaning